

{3, 1, 6, 5}

3	1	6	5
---	---	---	---

Array $\rightarrow$ a sequential collection of elements of the same data type.

`int arr[] = new int[4];`

index (indices) $\rightarrow$

0	1	2	3
4	9	5	2

$\rightarrow$ $\left\{ \begin{array}{l} \text{arr}[0] \\ \text{arr}[1] \\ \text{arr}[2] \\ \text{arr}[3] \end{array} \right.$

`arr[2] = 5;`

`arr[0] = 4;`

`arr[3] = 2;`

`arr[1] = 9;`

`System.out.println(arr[2]);` $\rightarrow$ 5

`System.out.println(arr[1]);` $\rightarrow$ 9

`arr[2] = 6;`

`System.out.println(arr[2]);` $\rightarrow$ 6

`System.out.println(arr[1]);` $\rightarrow$ 9

`arr[4] = 12;` $\rightarrow$ Array Index Out of Bounds Exception.

`arr.length` $\rightarrow$ 4
`int len = arr.length;`
index $\rightarrow$ 0 to (len-1)

`int arr[] = new int[n];` $\rightarrow$ $\left\{ \begin{array}{l} \text{arr}[0] \\ \text{arr}[1] \\ \text{arr}[2] \\ \vdots \\ \text{arr}[n-1] \end{array} \right.$

`System.out.println(arr.length);` $\rightarrow$ n

Q1) Given an array of n integers, find the sum of array.

$arr = \{3, 6, 19, 4\}$

$$\begin{aligned} sum &\rightarrow 3 + 6 + 19 + 4 \\ &= 32. \end{aligned}$$

```
int n = arr.length;
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
System.out.println(sum);
```

Q2) Given an array of n integers, find the max element of array.

$arr = \{3, 6, 19, 4\}$

$max \rightarrow 19.$

```
int n = arr.length;
X int max = 0;  $\rightarrow$  if all elements are -ve, then max remains 0.
for (int i = 0; i < n; i++) {
    max = Math.max(max, arr[i]);
}
System.out.println(max);
```

if ($max < arr[i]$)
 $max = arr[i];$

```
int n = arr.length;
✓ int max = arr[0]; / Integer.MIN_VALUE;
for (int i = 0; i < n; i++) {
    max = Math.max(max, arr[i]);
}
System.out.println(max);
```

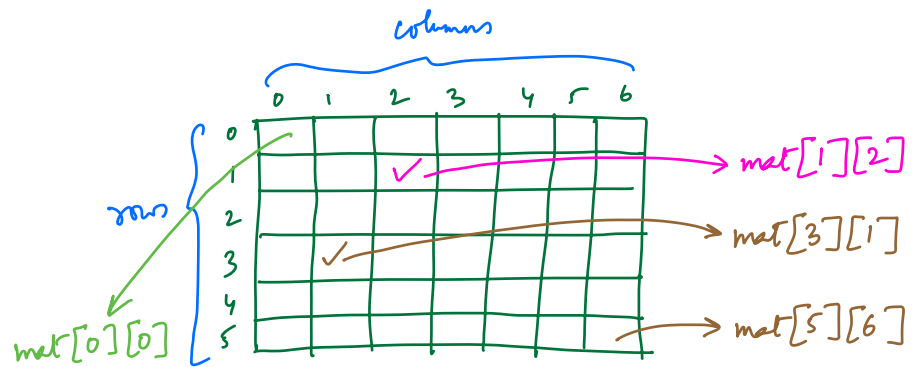
Q3) Given an array of n integers, find the min element of array.

```
int n = arr.length;
int min = arr[0]; / Integer.MAX_VALUE;
for (int i = 0; i < n; i++) {
    min = Math.min(min, arr[i]);
}
System.out.println(min);
```

[Break till 10:05 PM]

2D Arrays

- chessboard
- egg tray
- Sudoku
- Tic Tac Toe
- Theatre seats



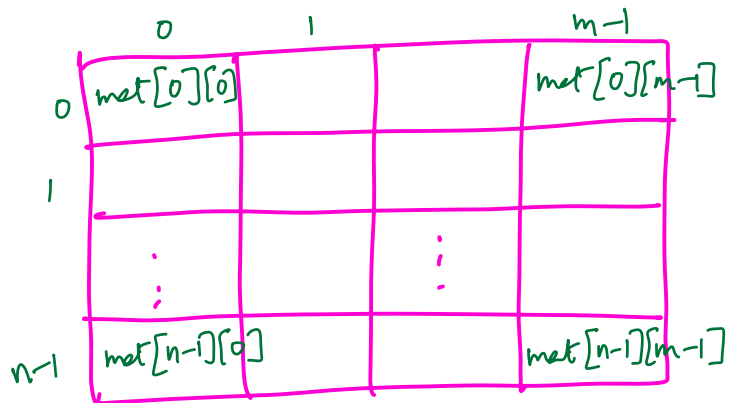
6 rows (0-5)

7 columns (0-6)

`int mat[][] = new int[6][7];`

n rows, m cols :-

`int mat[][] = new int[n][m];`

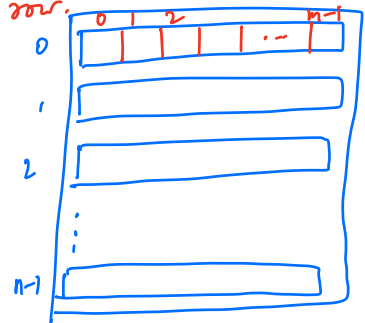


Q4) Given a matrix of size $n \times m$, print the top row.

Expected ans $\rightarrow$ $\text{mat}[0][0] \text{ mat}[0][1] \text{ mat}[0][2] \dots \text{mat}[0][m-1]$

```
int n = mat.length; // nr. of 1D arrays (n rows)
int m = mat[0].length; // nr. of elements in 0th row.
```

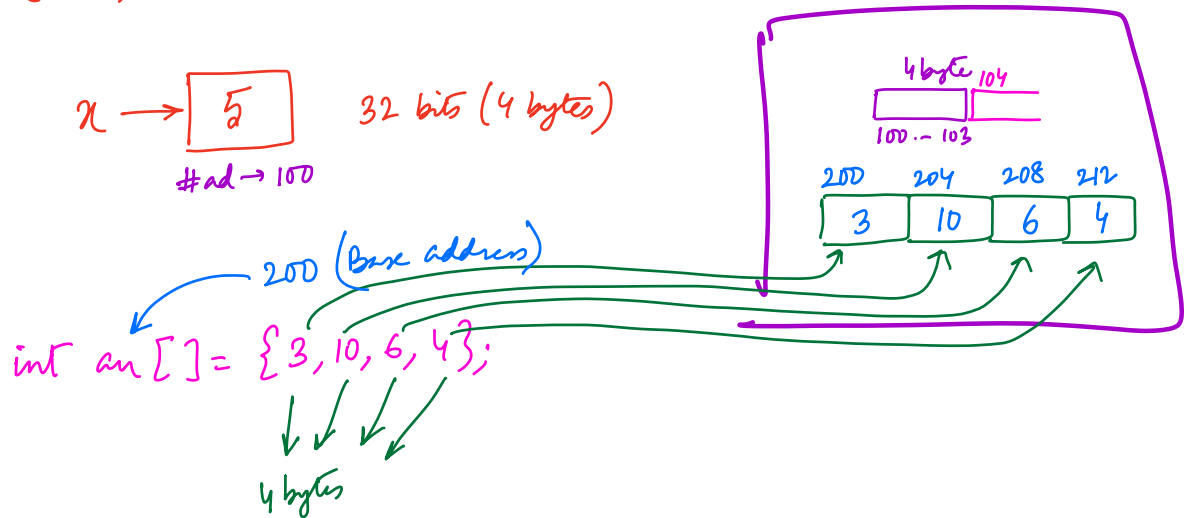
```
for (int j = 0; j < m; j++) {
    System.out.print(mat[0][j] + " ");
}
System.out.println();
```



```
Arrays.toString(mat[0]);
                1D array
```

```
int n = mat.length;
int m = mat[0].length;
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        System.out.print(mat[i][j] + " ");
    }
    System.out.println();
}
```

int x = 5;



Address of an[2] → Base address + 2 * (4 bytes)

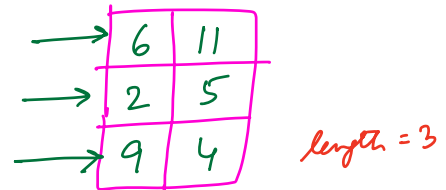
$$= 200 + 2 * 4$$

$$= \underline{\underline{208 \text{ bytes}}}$$

int mat[][] = new int[3][2];

mat[1] → Base Address +
1 * (size of mat[0])

$$= 300 + 1 * (2 * 4) = 308$$

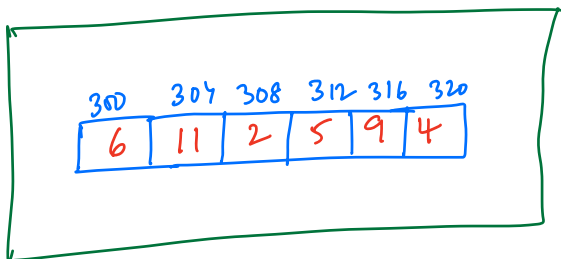


mat[2][1] → (Address of mat[2]) + offset (for index 1)

$$= [300 + 2 * (2 * 4)] + (1 * 4)$$

Memory

$$= 300 + 16 + 4 = 320$$



```
int an[] = {3, 10, 6, 4};
```

```
    .  
    <  
    .  
    .  
    }
```

```
an = new int[5]; // Array Redeclaration .
```

8 ↑ ff08
 ↓ ff10

ff08
ff09
ff0a
ff0b
ff0c
ff0d
ff0e
ff0f
ff10

int mat[][] = { {2, 3}, {4, 5}, {9, 8} }

mat[0] → {2, 3}
mat[1] → {4, 5}
mat[2] → {9, 8} } → mat.length → 3 .

mat[0].length = 2

mat[1].length = 2